

Objectifs : maîtriser les techniques avancées de conception et de développement de services web avec le framework Jakarta EE, en implémentant une API REST complète.

Ce projet consiste à concevoir un service de journal de bord, inspiré des applications de messagerie en ligne de type IRC telles que Mattermost, Slack, Discord, ou WhatsApp. Le service doit permettre de communiquer via des salons de discussion persistants (canaux), publics ou privés, accessibles via des URLs spécifiques. Commencez par lire l'intégralité de l'énoncé.

Phase 1 : Modèle de données

Jalon1. Identifiez les entités principales du système (utilisateur, canal, message, etc.) et leurs relations. Créez le **MCD** (Modèle Conceptuel de Données) correspondant aux données manipulées par le service.

Jalon2. Dérivez le **MLD** (Modèle Logique de Données) à partir du MCD. Identifiez les clés primaires et étrangères, et indiquez les contraintes d'intégrité référentielle.

Jalon3. Créez la **base de données** relationnelle et alimentez la avec des données jouets : au moins 3 utilisateurs, 2 canaux (un public, un privé), et une dizaine de messages. Vérifiez que vos données couvrent les cas d'usage principaux (canal public, canal privé).

Jalon4. Implémentez dans la **couche métier**, les classes Java représentant les entités et la logique applicative.

Jalon5. Implémentez la **couche de persistance** (DAOs) à l'aide de Jakarta Persistence (JPA) ou de JDBC. Chaque DAO expose les opérations CRUD nécessaires pour l'entité qu'il gère.

Phase 2 : API REST

Votre API REST doit prendre en charge :

- la récupération de la liste des messages d'un canal ;
- la création d'un message dans un canal ;
- la mise à jour d'un message uniquement par l'auteur ;
- la suppression d'un message par l'auteur ou l'administrateur du canal.

Jalon1. Concevez l'**API REST** du service en listant pour chaque ressource (canaux, messages, membres) les *end-points*, les méthodes HTTP, les codes de statut attendus et le format des corps de requête/réponse (JSON).

Jalon2. Implémentez les **servlets contrôleurs** (Jakarta Servlet) correspondant aux ressources du service qui retournent des codes HTTP appropriés et des messages d'erreur explicites en JSON.

Phase 3 : Client

Jalon1. Implémentez avec la technologie de votre choix (JavaScript/HTML, React, Angular, application mobile, etc.) un **client de messagerie** permettant de :

- s'authentifier et gérer sa session ;
- parcourir la liste des canaux publics disponibles et les rejoindre ;
- visualiser les messages d'un canal ;
- publier un message dans un canal ;
- modifier ou supprimer ses propres messages.

Le client doit consommer exclusivement votre API REST. Une interface minimaliste mais fonctionnelle est attendue.

Phase 4 : Authentification et sécurité

Jalon1. Pour garantir que seules les personnes autorisées puissent interagir avec les ressources du service, implémentez une **méthode d'authentification**.

Jalon2. Mettez en place un mécanisme de **contrôle d'accès** :

- seuls les membres d'un canal privé peuvent lire et écrire dans ce canal;
- seul l'auteur d'un message, voir l'administrateur du canal, peut le modifier ou le supprimer.

Phase 5 : Livraison

Jalon1. Le projet est à déposer sur Moodle avant la dernière séance. Le dépôt comprend :

- le code source complet (back-end et client);
- un fichier README.md décrivant l'architecture, les prérequis et les instructions de déploiement;
- un script SQL d'initialisation de la base de données avec des données exemples.